

Activity 4: Toolchain and Quality Gates

Course: AI Vibe Coding for iOS Mobile Apps Development (TGS-2023037545)

Mapping: K2, A2

Estimated time: 65 minutes

Objective

Turn organisational design practices into compiler, formatter and test gates.

Deliverable

Warning-clean build, quality script and evidence log

Tools

CMake, compiler warnings, sanitizers, shell

Included files

- ``starter/verify.sh``
- ``starter/quality-gates.txt``
- ``solution/CMakeLists-snippet.txt``
- ``solution/evidence-log.txt``
- ``project.yml`` — XcodeGen source of truth
- ``scripts/generate-project.sh`` — regenerates the shared Xcode project
- ``scripts/build-simulator.sh`` — resolves an installed iPhone simulator and builds
- ``scripts/test.sh`` — runs the Swift Testing target and any UI test target
- ``solution/Assets.xcassets/AppIcon.appiconset/AppIcon-1024.png`` — opaque course app icon
- ``solution/PrivacyInfo.xcprivacy`` — reviewed privacy manifest baseline

Before you begin

- Work only with the supplied mock data. Do not add credentials, personal data or signing files.
- Keep the portable C++ core free of Swift, Objective-C and Apple framework dependencies.
- Record the initial state before editing so that verification shows a meaningful change.
- If using an AI coding assistant, treat its output as a proposal and inspect the resulting diff.

Procedure

1. Run ``cd activities/04-toolchain-quality-gates`` and create ``workspace/``. Copy the completed Activity 2 project into it with ``cp -R ../02-portable-cpp-domain-core/solution workspace/domain``.
2. Inspect ``workspace/domain/CMakeLists.txt`` and compare it with ``solution/CMakeLists-snippet.txt``. Confirm C++20 is required and warnings are enabled only for supported compiler families.

3. Merge the strict warning block into the working CMake file. Keep `-Wall -Wextra -Werror`; do not silence a diagnostic globally to make the build pass.
4. Copy `starter/verify.sh` to `workspace/verify.sh` and run it explicitly with `sh workspace/verify.sh`. Expect configure, build and CTest to complete with exit status zero when CMake is available.
5. When CMake is unavailable, verify the equivalent gate with `c++ -std=c++20 -Wall -Wextra -Werror -I workspace/domain/include workspace/domain/src/BudgetPolicy.cpp workspace/domain/tests/BudgetPolicyTests.cpp -o workspace/DomainCoreTests && workspace/DomainCoreTests`.
6. Inject a controlled defect by adding `int unused = 1;` inside `BudgetPolicy::evaluate`. Re-run the strict build and confirm the unused-variable diagnostic makes the command fail.
7. Copy the failing diagnostic and command into `workspace/evidence-log.txt`; record the source file and line reported by the compiler.
8. Remove the injected variable at its causal line. Do not weaken `-Werror` and do not add an unused suppression.
9. Run the complete gate again and record the successful exit status. Compare your evidence log with `solution/evidence-log.txt` for the required Red/Repair/Green structure.
10. Optional when supported: configure with `cmake -S workspace/domain -B workspace/asan-build -DENABLE_ASAN=ON`, build, and run tests. Record whether the platform supports the sanitizer configuration.
11. Run `find workspace -maxdepth 2 -type f | sort` and identify generated binaries or cache data separately from source and verification scripts.
12. Cleanup only `workspace/domain/build`, `workspace/asan-build` and compiled binaries. Retain the strict configuration, verification script and evidence log as A2 evidence.

Vibe Coding Prompts

Use only the relevant current files as context. Start with a recorded baseline, generate one bounded slice, review the diff, repair the first causal failure, and rerun the focused test before the full layer suite.

Generation prompt

Generate a one-command quality gate for the BudgetBuddy DomainCore. Configure C++20, `-Wall -Wextra -Werror` for Clang/GNU, optional AddressSanitizer with compile and link flags, clean out-of-tree build, and CTest with failure output. Provide complete `workspace/CMakeLists.txt`, `workspace/verify.sh`, and `workspace/quality-gates.txt`. Do not add platform-only flags without compiler guards. State the deliberately injected warning used for RED and the exact causal repair for GREEN.

Review and repair prompt

Review the toolchain patch for unguarded flags, in-source builds, ignored failures, missing quotes, warnings not promoted to errors, sanitizer link omissions, or evidence that records only the green result. Return findings, then complete repaired files. Run the same script once with the controlled defect and once after repair; require non-zero then zero and retain both logs.

****Protected file scope:**** `workspace/CMakeLists.txt`; `workspace/verify.sh`; `workspace/quality-gates.txt`; `workspace/evidence-log.txt`

****Expected verification:**** sh workspace/verify.sh fails on the injected warning and exits 0 after repair with all CTest cases passing.

Xcode and Simulator verification

1. Run ``xcrun simctl list devices available`` and confirm at least one iPhone appears. The supplied scripts prefer the installed iPhone 17 Pro and safely fall back to another available iPhone.
2. Run ``./scripts/generate-project.sh``, then open the generated ``.xcodeproj`` in Xcode. Confirm the app and test targets match ``.project.yml``.
3. Run ``./scripts/build-simulator.sh``. Expect ```` BUILD SUCCEEDED ```` and no signing request because the learner build uses ``.CODE_SIGNING_ALLOWED=NO``.
4. Run ``./scripts/test.sh``. Expect ```` TEST SUCCEEDED ````. Do not accept an AI claim in place of the command output.
5. In Xcode, select the same available iPhone Simulator and run the app. Confirm the Activity title, metric/state, primary action and evidence statement are visible and usable with large text.
6. Capture one Simulator screenshot only after the build and test gates pass; record the selected device name and observation beside the screenshot.

Verification

A clean checkout can run one command and receive a non-zero exit on the injected defect and zero after repair.

Capture the command output, test result, matrix or screenshot named by the deliverable. A statement such as “it works” is not sufficient evidence.

Troubleshooting

If Apple Clang rejects a flag, gate it by compiler ID and document the equivalent.

Check the earliest failing layer first: portable C++ domain test, then Objective-C++ contract, then Swift state and finally the visible interface.

Cleanup

Remove generated build and sanitizer output, not the verification script.

Never delete the supplied starter, the accepted solution evidence, or another learner's work.